FINAL PROJECT REPORT

Metro Ticket Generating System in ServiceNow

Project Name	Metro Ticket Generating System in ServiceNow
Prepared By	Devi K
Platform	ServiceNow
Date	29 September 2026

Project Report Structure

- 1. Introduction
- 2. Ideation Phase
- 3. Requirement Analysis
- 4. Project Design
- 5. Project Planning & Scheduling
- 6. Functional and Performance Testing
- 7. Results
- 8. Advantages & Disadvantages
- 9. Conclusion
- 10. Future Scope
- 11. Appendix

1. INTRODUCTION

1.1 Project Overview

The Metro Ticket Generating System in ServiceNow is a digital ticket-booking and workflow-management prototype designed to streamline the metro ticketing process. Passengers use a ServiceNow Service Catalog item to select the source station, destination station, passenger type, and number of tickets. The solution calculates the fare, submits and processes the request through Flow Designer, stores ticket information in a custom Metro Database table, and presents a QR-based digital ticket for validation.

The solution is intentionally designed around ServiceNow's low-code capabilities. Flow Designer is used as the main automation engine, while UI Policies and essential Catalog Client Scripts support form behaviour. Roles and ACLs protect operational data, and Flow Designer Execution Details are used for troubleshooting and validation.

1.2 Purpose

- Reduce dependence on manual ticket counters and paper tickets.
- Provide a self-service digital ticket-booking experience.
- Automate fare processing and ticket-record creation.
- Provide a QR-based digital ticket for easier verification.
- Centralize booking information for metro operations.
- Create a maintainable ServiceNow solution that can be extended with payments, messaging and analytics.

2. IDEATION PHASE

2.1 Problem Statement

Metro passengers need a faster and more reliable way to book tickets, know the correct fare, and receive a verifiable digital ticket without depending entirely on physical counters or paper tickets. Metro operations teams simultaneously require centralized booking data, consistent fare processing, secure access, and an efficient method to verify ticket information.

The primary passenger problem is waiting time, manual effort, fare uncertainty and slower validation. The operational problem is fragmented or repetitive handling of ticket data and the need to trace workflow failures when automation does not behave as expected.

2.2 Empathy Map Canvas

Think & Feel	See	Hear
Wants fast booking, correct fare, immediate confirmation, and confidence that the QR ticket will be accepted.	Queues, physical ticketing, station staff, mobile-first services, and increasing QR-based interactions.	Announcements, staff instructions, commuter feedback, and expectations for faster digital public services.
Say & Do	**Pain**	**Gain**
Selects journey details, reviews fare, submits request, tracks status, opens QR ticket, and presents it for verification.	Waiting time, fare mismatch, missing data, portal rendering issues, unclear request status.	Faster self-service, reliable fare, digital ticket, centralized records, request tracking, reduced paper usage.

2.3 Brainstorming

- Passenger self-service booking through ServiceNow Service Portal.
- Automatic source-destination fare calculation.
- Structured ticket data stored in a custom table.
- Flow Designer-based automation instead of heavy custom code.
- QR-based digital ticket and station validation workflow.
- Role-based access, request tracking and maintainable administration.
- Future extensions: UPI/payment gateway, SMS/email ticketing, WhatsApp services and travel analytics.

The prioritized solution was the ServiceNow-based digital metro ticketing workflow because it provided high customer value while remaining feasible using the available platform capabilities.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	Passenger Experience	System Response
Need to Travel	Passenger decides to use a digital method to avoid queueing.	Provides a self-service digital alternative.
Open Portal	Passenger accesses ServiceNow Service Portal.	Loads the catalog item.
Enter Journey	Source, destination, passenger type and ticket count are entered.	Validates variables using policies/scripts.
Review Fare	System applies fare logic and displays/uses the calculated amount.	Applies fare/station mapping logic.
Submit Request	REQ/RITM is created and Flow Designer begins processing.	Flow Designer processes data and creates records.
Receive QR Ticket	Ticket data is rendered through the portal-compatible QR mechanism.	Widget/spModal presents digital ticket.
Station Validation	Passenger presents ticket; operational user verifies it.	Roles/ACLs control operational access.

Post Journey	Request/ticket record remains available for tracking and future analytics.	ServiceNow records retain booking history.

3.2 Solution Requirement

- FR-1: Provide a Service Catalog ticket-booking form.
- FR-2: Capture source, destination, passenger type and ticket quantity.
- FR-3: Calculate fare using configured route/fare logic.
- FR-4: Create a request and execute Flow Designer automation.
- FR-5: Map booking data into the custom Metro Database table.
- FR-6: Generate/display a QR-based digital ticket.
- FR-7: Provide request status tracking.
- FR-8: Support station/operations-side ticket validation.
- FR-9: Enforce role and ACL-based access control.
- FR-10: Support troubleshooting through Flow Execution Details.

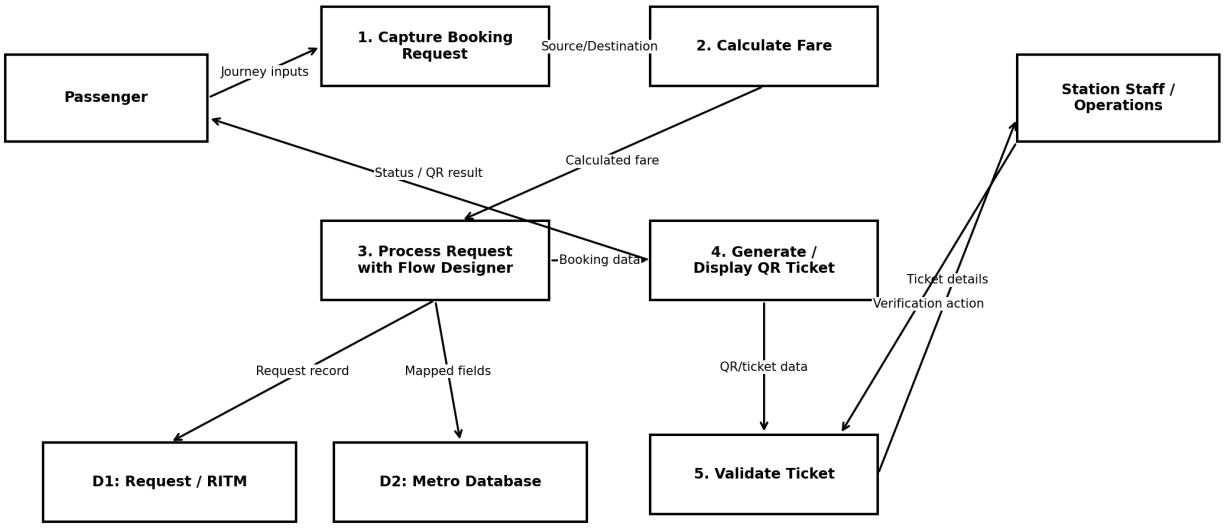
Non-functional requirements include usability, security, reliability, performance, availability, scalability, maintainability, data integrity, portal compatibility and auditability/troubleshooting.

3.3 Data Flow Diagram



DFD Level 0 - Context Diagram

Figure 1. DFD Level 0 - Passenger, Metro Ticket System and Metro Operations



DFD Level 1 - Metro Ticket Data Flow

Figure 2. DFD Level 1 - detailed booking, fare, flow, storage and QR data flow

3.4 Technology Stack

Component	Technology	Purpose
User Interface	ServiceNow Service Portal / Service Catalog	Passenger-facing booking experience.
Workflow Automation	ServiceNow Flow Designer	Processes submitted requests and downstream actions.
Data Storage	ServiceNow Request/RITM + custom Metro Database table	Stores request lifecycle and structured ticket data.
Client Behaviour	UI Policies + Catalog Client Scripts	Dynamic visibility, validation and field population.
Ticket Presentation	Service Portal widget + spModal	Renders QR-based digital ticket within portal constraints.
Security	Roles + ACLs	Controls access to booking and operational data.
Diagnostics	Flow Designer Execution Details / logs	Debugs failures and verifies execution paths.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The solution fits the identified customer problem because it moves the ticketing journey from a manual or fragmented process to a centralized digital workflow. Passengers receive a structured booking form, automated fare handling, digital status and QR ticketing, while metro operations receive standardized records and controlled access.

- Customer segment: metro passengers, station staff and metro operations.
- Jobs-to-be-done: book quickly, know correct fare, obtain proof of travel, verify ticket data, maintain accurate records.
- Constraints: portal compatibility, device/network availability, maintainability, implementation scope.
- Root causes: dependence on manual processes, fragmented data flow and lack of end-to-end automation.
- Solution fit: ServiceNow catalog + automation + structured storage + QR verification.

4.2 Proposed Solution

The proposed solution is an end-to-end ServiceNow-based metro ticketing prototype. It begins with a Service Catalog booking form and proceeds through fare calculation, Flow Designer, structured data storage, QR ticket generation, request tracking and operational verification.

- Novelty: ServiceNow is used as the full low-code workflow platform, not merely as a help-desk/request tool.
- Social impact: reduced queue dependency, reduced paper usage and improved commuter convenience.
- Operational impact: centralized records, traceable automation and easier handoff to administrators.
- Scalability: additional stations, fare bands, passenger types, integrations and analytics can be added incrementally.

4.3 Solution Architecture

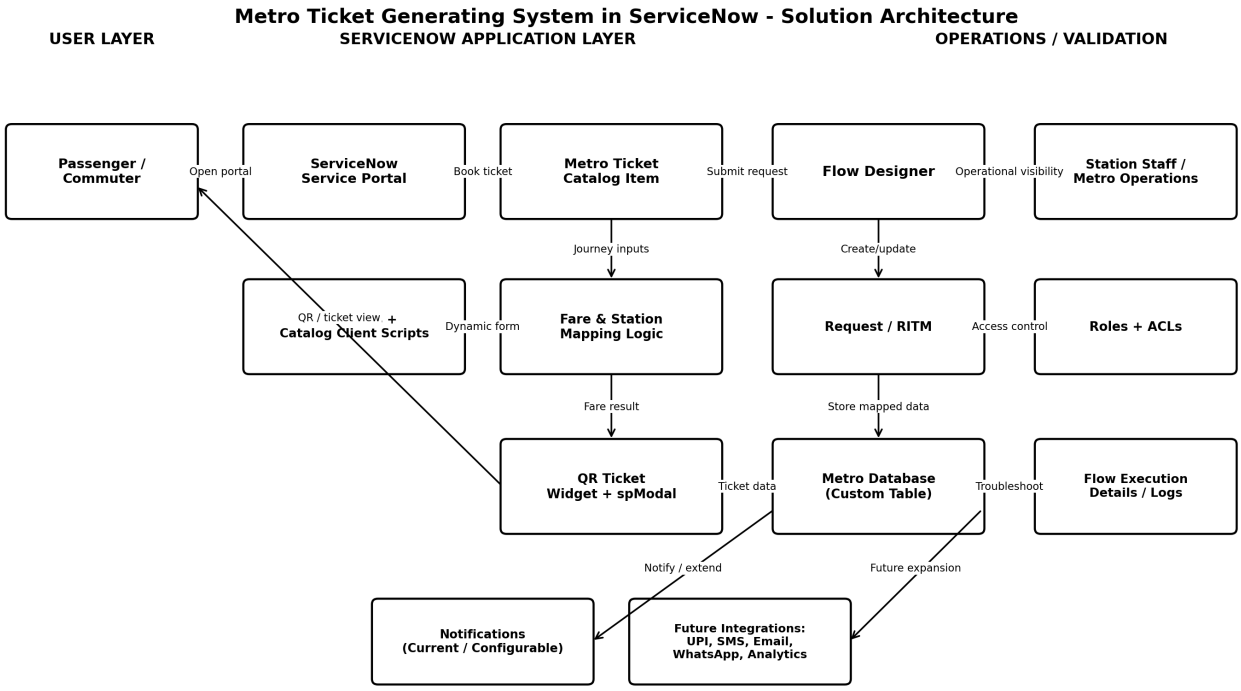


Figure 3. Solution architecture of the Metro Ticket Generating System

The architecture separates the passenger layer, ServiceNow application/workflow layer, data layer and operations/security layer. This separation supports maintainability and allows future payment, notification and analytics integrations without replacing the core booking workflow.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint	Epic / Focus	Story Points	Release Outcome
Sprint 1	Catalog & Form Setup	9	Catalog item, variables, UI policies and essential client-side behaviour.
Sprint 2	Fare Logic, Flow & Data	15	Fare/station logic, Flow Designer, request processing and custom-table mapping.
Sprint 3	QR, Portal & Access	13	QR ticket, widget/spModal, tracking, roles and ACLs.
Sprint 4	Testing, Debugging & Deployment	16	Fare/mapping debugging, flow diagnostics, QR testing, UAT, security and documentation.

Total planned/completed story points: 53. Average velocity across four sprints: 13.25 story points per sprint.

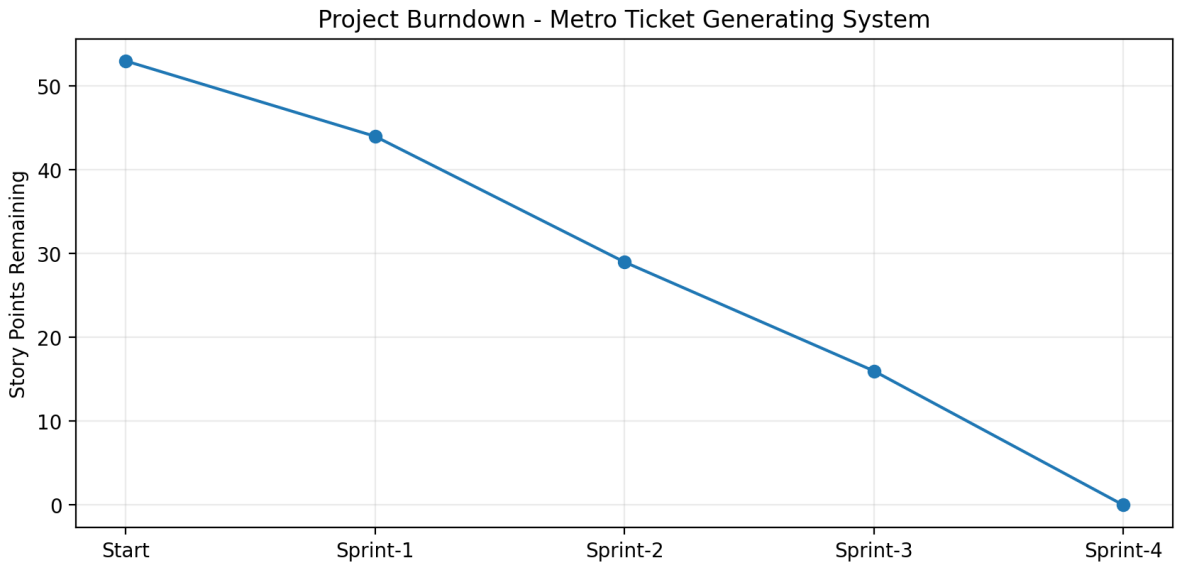


Figure 4. Project burndown based on the four-sprint planning model

6. FUNCTIONAL AND PERFORMANCE TESTING

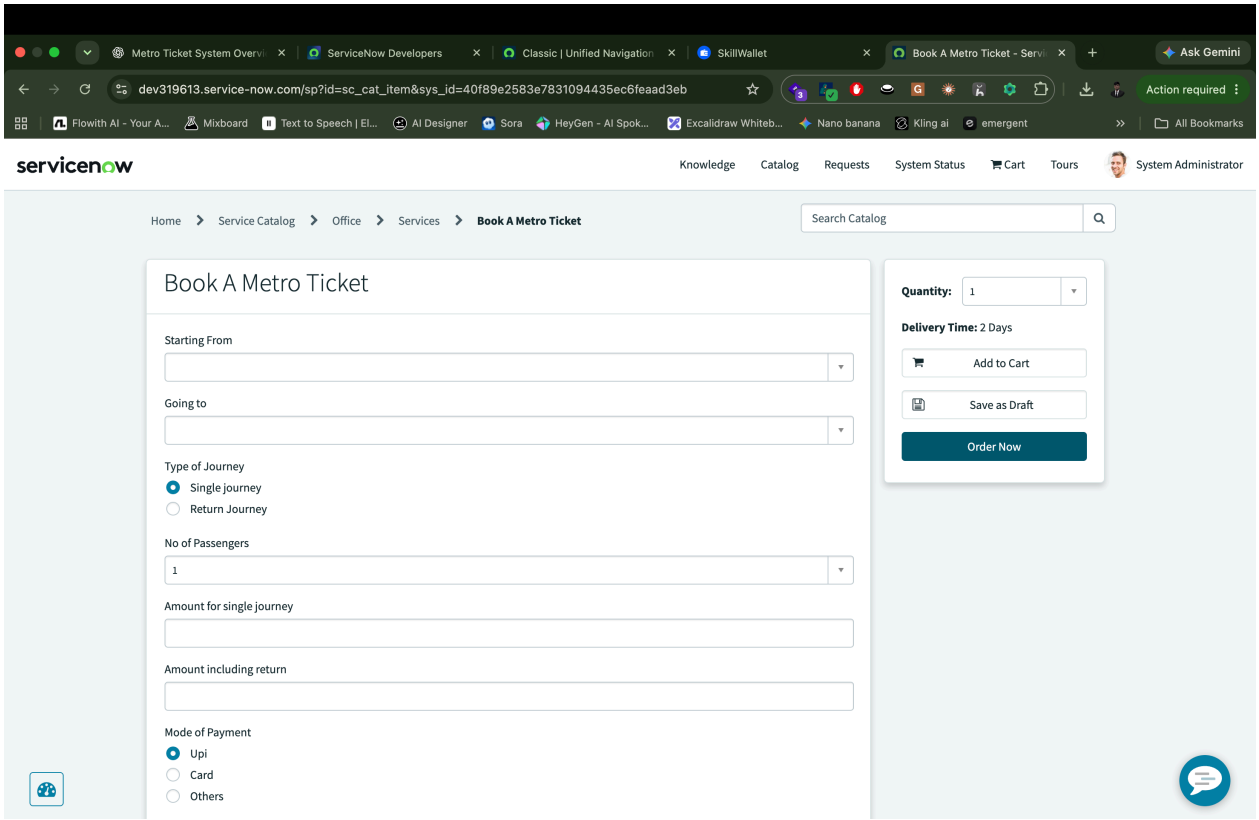
6.1 Performance Testing

Test ID	Test Scenario	Final Result
FT-01	Catalog and journey-input validation	Pass
FT-02	Passenger type and ticket-count validation	Pass
FT-03	Fare calculation	Pass after debugging fare mismatches
FT-04	Request submission and Flow Designer execution	Pass
FT-05	Variable-to-field mapping	Pass after correcting bindings
FT-06	QR ticket rendering	Pass after widget + spModal fix
FT-07	Request status tracking	Pass
FT-08	Role/ACL access	Pass
PT-01	Portal response behaviour	Pass - no abnormal delay reported in project testing
PT-02	Repeated request execution	Pass
PT-03	Portal compatibility / load behaviour	Pass

Machine-learning, GenAI, Power BI and Tableau model-performance metrics are not applicable to the current implementation because the solution does not use a trained AI/ML model or BI platform. These were documented as N/A instead of inventing accuracy or dashboard metrics.

7. RESULTS

7.1



Output Screenshots

The final implementation demonstrates the ServiceNow project workspace and the completed metro-ticketing workflow. The screenshot below shows the project context used during execution and documentation.

Figure 5. Metro Ticket Generating System project workspace

The implemented result includes a booking catalog item, dynamic fare handling, request processing through Flow Designer, structured storage in the Metro Database, QR-based digital ticket presentation, request tracking, role-controlled operational access and documented troubleshooting procedures.

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages / Current Limitations
Reduces manual ticketing effort and paper dependency.	Current project is a prototype rather than a production metro deployment.
Uses ServiceNow low-code automation for maintainability.	No integrated payment gateway/UPI in the current scope.
Centralizes passenger booking and ticket data.	SMS, email and WhatsApp ticket delivery are future scope.
Provides QR-based digital proof of booking.	No real-time external metro infrastructure integration is implemented.
Supports request tracking and operational troubleshooting.	No production performance/load benchmark figures were supplied.
Role/ACL security improves controlled access.	Portal/device/network conditions can affect digital access.
Architecture can be extended incrementally.	Travel analytics/dashboarding is not implemented yet.

9. CONCLUSION

The Metro Ticket Generating System demonstrates how ServiceNow can be adapted beyond conventional service-management use cases to implement a digital public-transport workflow. The project integrates passenger-facing catalog interaction, fare logic, Flow Designer automation, persistent data storage, QR ticket presentation, security and troubleshooting into one coherent process.

The most significant implementation challenges were QR rendering in Service Portal, fare mismatches, variable-to-field mapping errors and flow-execution issues. Resolving these problems strengthened the reliability of the final prototype and provided practical experience in ServiceNow portal behaviour, workflow debugging, low-code solution architecture and access control.

10. FUTURE SCOPE

- UPI, credit/debit card and wallet payment integration.
- Email, SMS and WhatsApp ticket delivery.
- Travel analytics and operational dashboards.
- Monthly passes, refunds and cancellation workflows.
- Real-time route/station information and service alerts.
- Offline-friendly ticket access and stronger mobile experience.
- NFC/smart-card integration where supported by the transport operator.
- Production-scale integration with metro AFC/validation infrastructure.